

Positional Encodings, Worked Out by Hand

The sinusoidal table, with a survey of learned, relative, ALiBi and RoPE

A complete numerical walk-through of the sinusoidal positional encoding the course builds in its `PositionalEncoding` module. We fix the model width to $d = 4$, write the two interleaved frequency channels in closed form, and fill in the exact 4×4 table of $\sin / \cos$ values for positions 0, 1, 2, 3. We then *add* it to a token-embedding block to show what the encoder actually consumes, explain the binary-clock geometry, and close with a one-line survey of the five position families modern Transformers use. Every number below was produced by the NumPy reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It takes the course formula $PE(pos, 2i) = \sin(pos/10000^{2i/d})$, $PE(pos, 2i+1) = \cos(pos/10000^{2i/d})$, specialises it to $d = 4$ (so there are exactly two frequencies, 1 and $\frac{1}{100}$), and computes every entry of the resulting position table by hand — then shows it being added, element-by-element, to a small embedding matrix.

How to read this. Each step isolates one mechanism: the frequency ladder, the $\sin / \cos$ interleave, the table, the addition, and the relative-shift property. Coloured boxes carry the intuition, the design trade-offs, and the traps. The final survey table is the map you keep — it tells you which scheme to reach for and who ships it.

Concepts covered, in order: why position must be injected at all · the geometric-frequency ladder $10000^{2i/d}$ · the $\sin / \cos$ interleave · the exact 4×4 PE table for $pos = 0 \dots 3$ · adding PE to embeddings · the binary-clock fingerprint and why relative shifts are linear maps · a survey of sinusoidal, learned-absolute, relative (T5), ALiBi and RoPE.

Internal learning material. Numbers are reproducible from the appendix code.

v1.0

1 Why position has to be injected at all

Self-attention computes, for every pair of tokens, a weighted sum over *all* tokens. Nothing in that computation looks at the order of the input: permute the tokens and the set of pairwise scores is the same set, just relabelled. Attention is, by construction, **order-agnostic** — it sees a bag of vectors, not a sequence.

So “the cat sat” and “sat the cat” would produce identical representations unless we *tell* the model where each token sits. The fix is to add a position-dependent vector to each token embedding before the first attention layer. The course does exactly this: `x = x + self.pe[:, :T, :]`. We now derive that `pe` table by hand.

Watch out

Position information is not optional decoration — it is load-bearing. Skip the positional add and a Transformer literally cannot distinguish word order; it degenerates to a permutation-invariant bag-of-tokens model. This is the single most important structural fact about Transformers versus RNNs: an RNN gets order *for free* from its sequential recurrence, attention does not.

2 Step 1 — The frequency ladder $10000^{2i/d}$

The encoding uses one $(\sin, \cos)$ pair per index $i = 0, 1, \dots, \frac{d}{2} - 1$, each pair oscillating at its own frequency. With model width $d = 4$ there are exactly two pairs, $i = 0$ and $i = 1$. The denominator that sets each frequency is

$$\boxed{\omega_i = \frac{1}{10000^{2i/d}}} \implies \begin{aligned} i = 0 : \quad & 10000^{0/4} = 10000^0 = 1, & \omega_0 &= 1, \\ i = 1 : \quad & 10000^{2/4} = 10000^{0.5} = 100, & \omega_1 &= \frac{1}{100} = 0.01. \end{aligned}$$

So the first pair turns at angular rate 1 radian per position (a *fast* wave) and the second at 0.01 radian per position (a very *slow* wave). For a real model with $d = 128$ the exponent $2i/d$ sweeps from 0 to nearly 1, laying down a smooth geometric sequence of wavelengths from 2π up to about $2\pi \cdot 10000$.

Intuition

Read the index i as “which clock hand.” Low dimensions (i small) are the *seconds* hand: high frequency, they flip quickly and pin down fine position. High dimensions (i large) are the *hour* hand: low frequency, they drift slowly and encode coarse, long-range position. Stack enough hands at geometrically spaced speeds and their joint reading is a unique fingerprint of the absolute position — a continuous analogue of a binary counter, where bit 0 flips every step, bit 1 every two steps, and so on.

3 Step 2 — The $\sin / \cos$ interleave

Each frequency contributes *two* adjacent columns: an even column holding the sine and the odd column right after it holding the cosine of the *same* angle:

$$PE(pos, 2i) = \sin(pos \cdot \omega_i), \quad PE(pos, 2i + 1) = \cos(pos \cdot \omega_i).$$

For $d = 4$ this fixes the meaning of all four columns:

$$\underbrace{\text{col } 0}_{\sin(pos \cdot 1)}, \underbrace{\text{col } 1}_{\cos(pos \cdot 1)} \quad \Bigg| \quad \underbrace{\text{col } 2}_{\sin(pos \cdot 0.01)}, \underbrace{\text{col } 3}_{\cos(pos \cdot 0.01)}.$$

Columns 0, 1 are the fast clock (frequency 1); columns 2, 3 are the slow clock (frequency 0.01). The reason to keep *both* $\sin$ and $\cos$ of each angle is the relative-shift property of Step 5: with the pair $(\sin \theta, \cos \theta)$ in hand, a shift $pos \rightarrow pos + k$ is a fixed rotation, which is what lets attention reason about *relative* distance.

4 Step 3 — The exact 4×4 table for $pos = 0, 1, 2, 3$

Now plug $pos \in \{0, 1, 2, 3\}$ into the four column formulas. Each entry is a single sin or cos evaluation; here are the angles and their values (radians, 3 d.p.):

$$\begin{aligned} \sin 0 &= 0.000, & \cos 0 &= 1.000, & \sin 0.00 &= 0.000, & \cos 0.00 &= 1.000, \\ \sin 1 &= 0.841, & \cos 1 &= 0.540, & \sin 0.01 &= 0.010, & \cos 0.01 &= 1.000, \\ \sin 2 &= 0.909, & \cos 2 &= -0.416, & \sin 0.02 &= 0.020, & \cos 0.02 &= 1.000, \\ \sin 3 &= 0.141, & \cos 3 &= -0.990, & \sin 0.03 &= 0.030, & \cos 0.03 &= 1.000. \end{aligned}$$

The slow-clock cosines all round to 1.000 and its sines are just $\approx pos \cdot 0.01$, because for tiny angles $\sin \theta \approx \theta$ and $\cos \theta \approx 1$. Assembling the columns row by row gives the full positional-encoding matrix:

$$PE = \begin{bmatrix} 0.000 & 1.000 & 0.000 & 1.000 \\ 0.841 & 0.540 & 0.010 & 1.000 \\ 0.909 & -0.416 & 0.020 & 1.000 \\ 0.141 & -0.990 & 0.030 & 1.000 \end{bmatrix}$$

Row pos is the vector added to the token at position pos . Read down column 0 ($\sin pos$): it swings $0 \rightarrow 0.841 \rightarrow 0.909 \rightarrow 0.141$ — the fast clock, already curving back. Read down column 2 ($\sin 0.01 pos$): it creeps $0 \rightarrow 0.010 \rightarrow 0.020 \rightarrow 0.030$ — the slow clock, barely moved. Two scales, encoded simultaneously, in one 4-vector per position.

Mini-example — this operation in isolation

The four entries of row $pos = 2$, computed from scratch:

$$(\sin 2, \cos 2, \sin 0.02, \cos 0.02) = (0.909, -0.416, 0.020, 1.000).$$

The fast clock has already passed its peak (sin falling, cos gone negative), while the slow clock has barely ticked off zero. One position, two readings at two scales — that contrast is the whole point.

5 Step 4 — It is *added* to the token embeddings

The encoder never sees PE alone. The token IDs are first looked up in an embedding table to produce a $(T \times d)$ block E , and then PE is added element-wise: $X = E + PE$. Take a concrete 4×4 embedding block (one row per token):

$$E = \begin{bmatrix} 0.500 & -0.200 & 0.100 & 0.300 \\ 0.400 & 0.400 & -0.100 & 0.200 \\ -0.300 & 0.100 & 0.500 & 0.000 \\ 0.200 & -0.500 & 0.300 & 0.400 \end{bmatrix}.$$

Adding the PE matrix from Step 3 entry by entry (e.g. row 1, col 0: $0.400 + 0.841 = 1.241$; row 3, col 1: $-0.500 + (-0.990) = -1.490$) yields what the first attention layer actually receives:

$$X = E + PE = \begin{bmatrix} 0.500 & 0.800 & 0.100 & 1.300 \\ 1.241 & 0.940 & -0.090 & 1.200 \\ 0.609 & -0.316 & 0.520 & 1.000 \\ 0.341 & -1.490 & 0.330 & 1.400 \end{bmatrix}.$$

Same token embedding in two different positions now produces two different input rows — order has been injected, additively, with no new parameters.

Watch out

The positional table is added, not concatenated. That means the embedding dimension d is shared between “what the token is” and “where it sits” — they superimpose. This works because the network learns to read the two signals out of the sum, but it also means the embedding magnitudes and the PE magnitudes (each component in $[-1, 1]$) should be on comparable scales; this is part of why Transformers scale embeddings by $\sqrt{d}$ before the add.

6 Step 5 — The binary-clock fingerprint and linear shifts

Why $\sin$ and $\cos$ of the same angle? Because a shift in position then becomes a **rotation** of the $(\sin, \cos)$ pair — a fixed linear map that does not depend on the absolute position. For the fast clock (frequency 1), advancing one position rotates the pair by exactly 1 radian:

$$\begin{bmatrix} \sin(pos + 1) \\ \cos(pos + 1) \end{bmatrix} = \underbrace{\begin{bmatrix} \cos 1 & \sin 1 \\ -\sin 1 & \cos 1 \end{bmatrix}}_{R(1)} \begin{bmatrix} \sin pos \\ \cos pos \end{bmatrix}, \quad R(1) = \begin{bmatrix} 0.540 & 0.841 \\ -0.841 & 0.540 \end{bmatrix}.$$

Check it against the table. Take $pos = 1$, pair $(\sin 1, \cos 1) = (0.841, 0.540)$, and apply $R(1)$:

$$R(1) \begin{bmatrix} 0.841 \\ 0.540 \end{bmatrix} = \begin{bmatrix} 0.540 \cdot 0.841 + 0.841 \cdot 0.540 \\ -0.841 \cdot 0.841 + 0.540 \cdot 0.540 \end{bmatrix} = \begin{bmatrix} 0.909 \\ -0.416 \end{bmatrix} = \begin{bmatrix} \sin 2 \\ \cos 2 \end{bmatrix} \cdot \checkmark$$

The same rotation matrix $R(1)$ maps row 0 $\rightarrow$ row 1 $\rightarrow$ row 2 $\rightarrow$ row 3. So the model can compute a *relative* offset of k as one fixed linear operator, identical everywhere in the sequence — it never has to memorise each absolute position separately.

Intuition

Picture each frequency pair as a point on the unit circle, the angle being $pos \cdot \omega_i$. Moving forward k positions spins every point by a fixed angle $k\omega_i$ — fast hands spin a lot, slow hands barely move. The joint configuration of all the hands is a unique fingerprint of the absolute position (like a clock face), yet *differences* between two positions are encoded by the same rotation regardless of where they sit. Absolute identity and relative distance, both for free, from one trig table.

7 The five position families, at a glance

Sinusoidal is the original recipe; production systems have since branched. Here is the map. $L_{\max}$ is the maximum length the scheme was trained on.

Family	One-line mechanism	Who uses it
Sinusoidal (fixed)	add fixed $\sin / \cos$ table to embeddings; no params	original Transformer
Learned absolute	add a trainable row per position; learned embedding	BERT, GPT-2
Relative (T5 bias)	add a learned scalar bias per relative distance to scores	T5, Transformer-XL
ALiBi (linear)	subtract $m \cdot \text{dist}$ from each attention score	BLOOM, MPT
RoPE (rotation)	rotate q, k by an angle $\propto \text{pos}$ before the dot	LLaMA, GPT-NeoX, most LLMs

Two of these are worth a number. **ALiBi** gives head h a fixed slope (for 8 heads, $m = 0.5, 0.25, 0.125, \dots, 0.004$) and penalises a score by $-m \cdot \text{dist}$: with $m = 0.5$ the penalties at distance 0, 1, 2, 3 are 0.0, -0.5 , -1.0 , -1.5 — a built-in recency bias that extrapolates to lengths never trained on. **RoPE** rotates the query and key vectors by angles $m\theta$ and $n\theta$; the resulting dot product depends *only* on the relative offset $n - m$. In the reference code, $(m, n) = (2, 5), (3, 6), (0, 3)$ all share offset 3 and all give the identical score -0.990 — relative position, baked into the geometry.

When this is the right choice

Use **sinusoidal** or **learned-absolute** when the sequence length is fixed and modest (classifiers, encoders like the course’s log-Transformer). Reach for **ALiBi** when you need cheap length extrapolation with almost no machinery. Use **RoPE** for any modern decoder-only LLM — it is the de-facto default (LLaMA, Mistral, Qwen, GPT-NeoX) because it injects relative position directly into attention and extrapolates gracefully.

Watch out

Learned tables hard-fail past their trained length. A learned-absolute embedding has exactly $L_{\max}$ rows; ask for position $L_{\max}$ or beyond and there is simply no vector to look up — an index error, or silent garbage if clamped. Sinusoidal and RoPE are defined by a *formula*, so they evaluate at any position, but even they degrade out-of-distribution if the model never saw long contexts. Length generalisation is governed by the position scheme as much as by the data.

8 Equation cheat-sheet

Frequency of pair i	$\omega_i = 1/10000^{2i/d}$
Even column (sine)	$PE(pos, 2i) = \sin(pos \cdot \omega_i)$
Odd column (cosine)	$PE(pos, 2i + 1) = \cos(pos \cdot \omega_i)$
$d = 4$ frequencies	$\omega_0 = 1, \omega_1 = 0.01$
Injection into encoder	$X = E + PE$ (element-wise add)
Relative shift (per pair)	$\begin{bmatrix} \sin(pos + k) \\ \cos(pos + k) \end{bmatrix} = R(k\omega_i) \begin{bmatrix} \sin pos \\ \cos pos \end{bmatrix}$
Rotation matrix	$R(\phi) = \begin{bmatrix} \cos \phi & \sin \phi \\ -\sin \phi & \cos \phi \end{bmatrix}$
ALiBi score penalty	score $\neg = m_h \cdot i - j $
RoPE property	$\langle R_m q, R_n k \rangle$ depends only on $n - m$

Appendix A Reference implementation

Every number above is reproducible from the following program (NumPy; the course's `PositionalEncoding` module computes the identical table in PyTorch). Change `d` or the positions to explore.

```
import numpy as np
np.set_printoptions(precision=3, suppress=True)

d = 4                                # model width -> 2 frequency pairs
pos = np.arange(4)                   # positions 0,1,2,3

# frequencies: omega_i = 1 / 10000**(2i/d), for i = 0 .. d/2-1
div = 10000.0 ** (-np.arange(0, d, 2) / d)
print("frequencies:", div)           # [1.  0.01]

# build the PE table: even cols = sin, odd cols = cos
PE = np.zeros((4, d))
for p in pos:
    PE[p, 0::2] = np.sin(p * div)    # cols 0,2
    PE[p, 1::2] = np.cos(p * div)    # cols 1,3
print(np.round(PE, 3))
# [[ 0.  1.  0.  1. ]
# [ 0.841 0.54 0.01 1. ]
# [ 0.909 -0.416 0.02 1. ]
# [ 0.141 -0.99 0.03 1. ]]

# it is ADDED to token embeddings: X = E + PE
E = np.array([[ 0.5, -0.2, 0.1, 0.3],
               [ 0.4, 0.4, -0.1, 0.2],
               [-0.3, 0.1, 0.5, 0.0],
               [ 0.2, -0.5, 0.3, 0.4]])
print(np.round(E + PE, 3))
# [[ 0.5  0.8  0.1  1.3 ]
# [ 1.241 0.94 -0.09 1.2 ]
# [ 0.609 -0.316 0.52 1. ]
# [ 0.341 -1.49 0.33 1.4 ]]

# relative shift is a rotation: R(1) maps row pos -> row pos+1 (fast clock)
c1, s1 = np.cos(1), np.sin(1)
R = np.array([[c1, s1], [-s1, c1]])    # rotate (sin,cos) by +1 rad
print(np.round(R @ np.array([s1, c1]), 3)) # [ 0.909 -0.416] == (sin2, cos2)

# RoPE: dot product of rotated q,k depends only on relative offset n - m
theta = 1.0
def rope(x, p):
    c, s = np.cos(p*theta), np.sin(p*theta)
    return np.array([x[0]*c - x[1]*s, x[0]*s + x[1]*c])
q = k = np.array([1.0, 0.0])
for m, n in [(2, 5), (3, 6), (0, 3)]:    # all have offset 3
    print(m, n, round(float(rope(q, m) @ rope(k, n)), 3)) # all -0.99
```

— end of document —